

Road Sign Detection

In this lesson, the road sign recognition can be achieved through the commands.

1. Getting Ready

- 1) Before starting, ensure the map is laid out flat and free of wrinkles, with smooth roads and no obstacles. For specific map laying instructions, please refer to "Map Laying and Prop Installation" in the same directory as this section for guidance.
- 2) The roadmap model discussed in this section is trained using YOLOv5. For further information on YOLOv5 and related content, please refer to "16. Machine Learning".
- 3) When experiencing this game, ensure it is conducted in a well-lit environment, but avoid direct light shining on the camera to prevent misrecognition.

2. Program Logic

Firstly, acquire the real-time image from the camera and apply operations such as erosion and dilation.

Next, invoke the YOLOv5 model and compare it with the target screen image. Finally, execute the appropriate landmark action based on the comparison results.

You can find the source code for this program at:

```
/home/ubuntu/ros2_ws/src/example/example/yolov5_detect/yolov5_trt.py
```

3. Operation Steps

Note: The following steps exclusively activate road sign detection in the live camera feed,

without executing associated actions. Seeking direct experience with autonomous driving, you may bypass this lesson and proceed to "Integrated Application" within the same file. When inputting commands, please ensure accurate differentiation between uppercase and lowercase letters as well as spaces. Utilize the "Tab" key for keyword completion.

- 1) Start the robot, and access the robot system desktop using VNC.



- 2) Click-on  to open the command-line terminal.

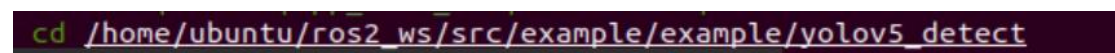
- 3) Execute the command to disable the app auto-start service.

```
~/stop_sh
```



- 4) Run the command to navigate to the directory containing programs.

```
cd /home/ubuntu/ros2_ws/src/example/example/yolov5_detect
```



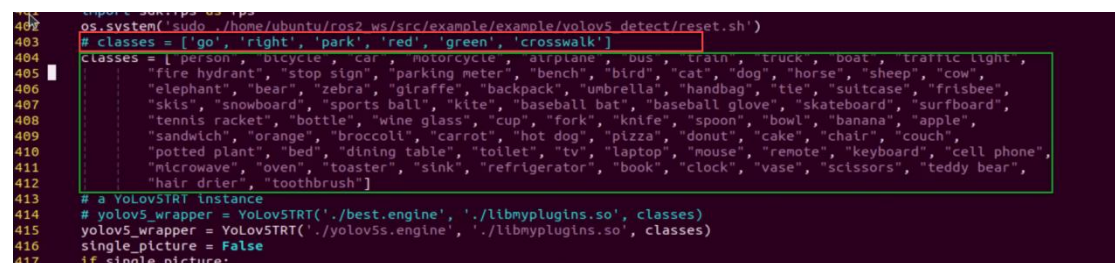
- 5) Type the command to open the program source code.

```
vim yolov5_trt.py
```



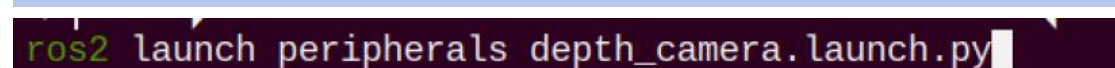
- 6) Press the "i" key to enter insertion mode. Find the code enclosed in the red box and toggle commenting by adding or removing comments as needed.

Once done, press the "ESC" key, then type ":wq" and press Enter to save and exit.



- 7) Run the command to open the camera node.

```
ros2 launch peripherals depth_camera.launch.py
```



- 8) Open a new command line terminal. Enter the command to enable the camera node.

```
ros2 launch yolov5_ros2 yolov5_ros2.launch.py  
model:="traffic_signs_640s_7_0"
```

```
/bin/zsh 86x22  
-----  
当前环境是ROS2|The current environment is ROS2  
-----  
LIDAR: LD19  
CAMERA: ascamera  
MACHINE: MentorPi_Mecanum  
HOST: /  
MASTER: /  
ROS_DOMAIN_ID: 0  
VERSION: |V1.0.0|2024-09-06|  
zsh: corrupt history file /home/ubuntu/.zsh/.zsh_history  
07:40:23  
ros2 launch yolov5_ros2 yolov5_ros2.launch.py model:="traffic_signs_640s_7_0"
```

- 9) Run the command to initiate game program.

```
ros2 launch peripherals depth_camera.launch.py
```

```
-----  
当前环境是ROS2|The current environment is ROS2  
-----  
LIDAR: LD19  
CAMERA: ascamera  
MACHINE: MentorPi_Mecanum  
HOST: /  
MASTER: /  
ROS_DOMAIN_ID: 0  
VERSION: |V1.0.0|2024-09-06|  
zsh: corrupt history file /home/ubuntu/.zsh/.zsh_history  
07:40:23  
ros2 launch peripherals depth_camera.launch.py
```

- 10) Open a new command line terminal. Execute the command to open the interface for live camera feed.

```
rqt
```

```
> rqt  
QStandardPaths: XDG_RUNTIME_DIR not set, defaulting to '/tmp/runtime-ubuntu'
```

- 11) Position the road sign in front of the camera, and the robot will recognize the road sign automatically. If you need to terminate this game, press "Ctrl+C". If it fails, please try again.

Note: In the event that the model struggles to recognize traffic-related signs, it

may be necessary to lower the confidence level. Conversely, if the model consistently misidentifies traffic-related signs, raising the confidence level might be advisable.

12) Run the command to navigate to the directory containing programs.

```
cd /home/ubuntu/ros2_ws/src/example/example/self_driving
```

```
cd /home/ubuntu/ros2_ws/src/example/example/self_driving
```

13) Execute the command to open the game program.

```
vim self_driving.launch.py
```

```
vim self_driving.launch.py
```

The red box represents the confidence value, which can be adjusted to modify the effectiveness of target detection.

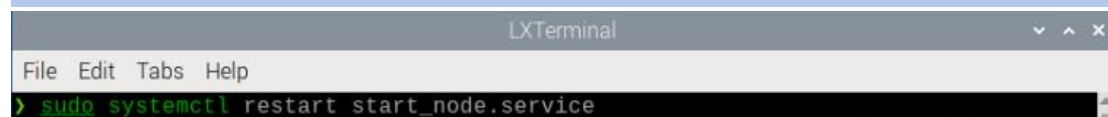
```
90 )
91
92 yoloV5_node = Node(
93     package='example',
94     executable='yoloV5_node',
95     output='screen',
96     parameters=[{'classes': ['go', 'right', 'park', 'red', 'green', 'crosswalk']},
97                 {'use_depth': 'true', 'engine': 'traffic_signs_640s_7_0.engine', 'lib': 'libmyplugins.so', 'conf': 0.75}],
98 )
99
100 self_driving_node = Node(
101     package='example',
102     executable='self_driving',
103     output='screen',
104     parameters=[{'start': start}, {'only_line_follow': only_line_follow}],
105 )
```

Once you've completed the game experience, you can initiate the app service by following the instructions or restarting the robot.

If the app service is not activated, the associated app functions will be disabled. (The app service will automatically start when the robot restarts).

To restart the app service, enter the following command and wait for the buzzer to emit a single beep, indicating that the service startup is complete.

```
sudo systemctl restart start_node.service
```



```
LXTerminal
File Edit Tabs Help
> sudo systemctl restart start_node.service
```

4. Program Outcome

After initiating the game, place the robot on the road within the map. Once the robot identifies landmarks, it will highlight the detected landmarks and annotate them based on the highest confidence level learned from the model.

